

MATH3311-MATH5335, Lab Test, Duration: 90 minutes, Term 2, 2025

Question 1.

For Python users:

1- Copy the variables **sumA**, **detA**, **minEV**, **maxEV**, **SumsvA**, **comment1**, **comment2**, **comment3** into Matlab Grader together with your Python code.

2- Import the following:

```
from numpy import ones, sum, min, max
from numpy.linalg import det
from scipy.linalg import svd, lu, eigvals
```

Let n be a positive integer and consider the n -by- n matrix

$$A = [a_{i,j}]_{1 \leq i,j \leq n} = \begin{pmatrix} a_{1,1} & \beta & \beta & \cdots & \beta & \beta \\ \beta & a_{2,2} & \beta & \cdots & \beta & \beta \\ \beta & \beta & a_{3,3} & \cdots & \beta & \beta \\ \vdots & \vdots & \vdots & \cdots & & \\ \beta & \beta & \beta & \cdots & \beta & a_{n,n} \end{pmatrix}$$

where $\beta = \frac{1}{3}$ and $a_{i,i} = \frac{1}{i}$ for $1 \leq i \leq n$.

Fix $n=10$.

(a) Generate the matrix A efficiently. Compute the sum of all entries of the matrix A and store the value in the variable **sumA**.

(b) Compute the determinant of the matrix A and store the value in the variable **detA**.

(c) Compute the minimum eigenvalue and the maximum eigenvalue of A and store the values in the variables **minEV** and **maxEV**, respectively.

(d) Is A positive-definite? If yes, type **comment1 = 'yes'**, otherwise, type **comment1 = 'no'**.

(e) Is A nonsingular? If yes, type **comment2 = 'yes'**, otherwise, **comment2 = 'no'**.

(f) Use the SVD-decomposition to determine the singular values of the matrix A . Compute the sum of the singular values of A and store the values in the variable **SumsvA**.

(g) Is the row partial pivoting (Permutation matrix P) needed when performing LU factorization of the matrix A ? Type **comment3 = 'yes'** if P is needed or **comment3 = 'no'** if P is not needed.

Matlab code:

```

n = 10;
A = ones(n, n) * (1/3);
for i = 1:n
    A(i, i) = 1 / i;
end
sumA=sum(sum(A));
minEV=min(eig(A));
maxEV=max(eig(A));
detA=det(A);
SumsvA=sum(svd(A));
[L,U,P]=lu(A);

```

Python code:

```

from numpy import ones, sum, min, max
from numpy.linalg import det
from scipy.linalg import svd, lu, eigvals

n = 10
A = ones((n, n)) * (1 / 3)
for i in range(n):
    A[i, i] = 1 / (i + 1)
sumA = sum(A)
detA = det(A)
EV = eigvals(A)
minEV = min(EV)
maxEV = max(EV)
svs = svd(A, compute_uv=False)
SumsvA = sum(svs)
P, L, U = lu(A)

```

comment1 = 'no'; % Because A is symmetric, but not all eigenvalues are positive.

comment2 = 'yes'; % Because detA is not zero,

comment3 = 'no'; % Because P is the identity matrix

Question2.

For Python users: 1- Copy the variables **x1, x2, xe, CR, q** into Matlab Grader together with your Python code.

2- Import the following: `from numpy import sqrt, sin, exp, cos, log`

The Newton's method is an iterative technique for estimating a root of a nonlinear equation $g(x) = 0$, i.e. a value x^* such that $g(x^*) \approx 0$. It is defined as:

$$x_{n+1} = x_n - \frac{g(x_n)}{g'(x_n)}, \quad n = 0, 1, 2, 3, \dots$$

Let $f(x) = \sqrt{4-x} + \sin(x) + \exp(x)$ for $x \in [0,3]$.

The aim is to find (estimate) the value of q in the interval $[0,3]$ using Newton's method such that $f(q)=8$.

a) Define a variable x_0 and set it to 0.7. Use this value as the initial guess for the Newton iteration.

b) Modify the Newton's method above so that it can be used to find (estimate) the point q in the interval $[0,3]$. Write a Matlab\Python program which uses this modified Newton iteration. Compute x_1 and x_2 , and store the results in **x1** and **x2**, respectively.

c) One way to estimate the convergence rate of Newton's method is to compute

$$\frac{\log\left(\left|\frac{e_{n+1}}{e_n}\right|\right)}{\log\left(\left|\frac{e_n}{e_{n-1}}\right|\right)}$$

for some values of n , where $e_n = x_n - x_{n-1}$ is the difference between successive approximations.

Use $n=4$ to estimate the convergence rate and store the results in the variable **CR**.

d) Run the Newton iteration till the difference between successive approximations is smaller 10^{-2} , i.e. $|x_{n+1} - x_n| < 10^{-2}$. Store the result in **xe**.

e) Find q and store your result in the

Matlab code:

```
g = @(x)(4-x).^(1/2) + sin(x) + exp(x) - 8;
dg = @(x) -(1/2)*(4-x).^(-1/2) + cos(x) + exp(x);

x0 = 0.7;
xx(1)=x0;
for i=1:6;
xx(i+1) = xx(i) - g(xx(i))/dg(xx(i));
E(i)=xx(i+1)-xx(i);
end
x1 = xx(2);
x2 = xx(3);
CR = log(abs(E(5)/E(4)))/log(abs(E(4)/E(3)));
% find xe
x_prev = x0;
x_curr = x1;
while abs(x_curr - x_prev) >= 1e-2
    g_curr = g(x_curr);
    dg_curr = dg(x_curr);
    x_next = x_curr - g_curr/dg_curr;
    x_prev = x_curr;
    x_curr = x_next;
end
xe = x_curr;
```

Python code:

```
from numpy import sqrt, sin, exp, cos, log
def g(x):
    return sqrt(4 - x) + sin(x) + exp(x) - 8
def dg(x):
    return -0.5 * (4 - x)**(-0.5) + cos(x) + exp(x)
x0 = 0.7
xx = [x0]
E = []
for i in range(6):
    x_new = xx[i] - g(xx[i]) / dg(xx[i])
    xx.append(x_new)
    E.append(xx[i+1] - xx[i])
x1 = xx[1]
x2 = xx[2]
CR = log(abs(E[4] / E[3])) / log(abs(E[3] / E[2]))
# find xe
x_prev = x0
x_curr = x1
while abs(x_curr - x_prev) >= 1e-2:
    g_curr = g(x_curr)
    dg_curr = dg(x_curr)
    x_next = x_curr - g_curr / dg_curr
    x_prev = x_curr
    x_curr = x_next
xe = x_curr
```

You still need to find q .

Question 3.

For Python users: 1- Copy the variables **N, x, y1, y2, f1, f2, Qf, errQ** into Matlab Grader together with your Python code.

2- Import the following: `from numpy import sqrt, arctan, log, linspace`

The goal is to approximate the integral $I := \int_0^2 (\ln(1 + x^2) + 5)dx$ using a composite two-point Gauss–Legendre quadrature rule.

The composite two-point Gauss quadrature rule for an integrand g is defined as:

$$\int_a^b g(x)dx \approx \frac{h}{2} \sum_{i=0}^{N-1} \left(g(x_i + h\xi_1) + g(x_i + h\xi_2) \right),$$

N is the number of subintervals, $h = \frac{b-a}{N}$, $x_i = a + i h$, for $0 \leq i \leq N - 1$, $\xi_1 = \frac{1}{2} \left(1 - \frac{1}{\sqrt{3}} \right)$ and $\xi_2 = \frac{1}{2} \left(1 + \frac{1}{\sqrt{3}} \right)$.

a) Set $N=6$ and store it in the variable **N** and define a variable **h** given by $h=(b-a)/N$.

b) Compute an approximation to the integral I using the composite two-point Gauss–Legendre quadrature rule in the following way:

Define a row vector **x** of N equally spaced points whose first component is 0 and whose last component is $(N-1)h$.

Define the row vectors **y1** = $\mathbf{x} + h\xi_1$ and **y2** = $\mathbf{x} + h\xi_2$ of length N each.

Compute the function $f(x) = \ln(1 + x^2) + 5$ at the entries of the row vector **y1** and store your results in the row vector variable **f1**. Note: $\ln$ is the natural logarithmic function. Hint: $\log(z)$ computes $\ln(z)$ in both Matlab and Python.

Compute the function $f(x) = \ln(1 + x^2) + 5$ at the entries of the row vector **y2** and store your results in the row vector variable **f2**.

Store the value of this approximation in the variable **Qf**.

c) Use the true solution $2 \ln(5) + 6 + 2 \arctan(2)$ to compute the absolute errors from composite Gauss–Legendre quadrature rule. Store the absolute error in **errQ**. The function $\arctan$ is the inverse tangent. Hint: $\text{atan}(z)$ computes $\arctan(z)$ in Matlab and $\arctan(z)$ computes $\arctan(z)$ in Python.

Matlab code:

```

N = 6;
h = 2/N;
x = linspace(0,(N-1)*h,N);
xi1=1/2*(1-1/sqrt(3));
xi2=1/2*(1+1/sqrt(3));
y1=x+h*xi1;
y2=x+h*xi2;
f1=log(1+y1.^2)+5;
f2=log(1+y2.^2)+5;
Qf = h/2*(sum(f1)+sum(f2));
trsol =
2*log(5)+2*atan(2)+6;
errQ = abs(Qf - trsol)

```

Python code:

```

from numpy import sqrt, arctan, log, linspace
N = 6;
h = 2/N;
x = linspace(0,(N-1)*h,N);
xi1=1/2*(1-1/sqrt(3));
xi2=1/2*(1+1/sqrt(3));
y1=x+h*xi1;
y2=x+h*xi2;
f1=log(1+y1**2)+5;
f2=log(1+y2**2)+5;
Qf = h/2*(sum(f1)+sum(f2));
trsol = 2*log(5)-4+2*arctan(2)+10;
errQ = abs(Qf - trsol)

```

Question 4.

For Python users: 1- Copy the variables **tdata**, **ydata**, **yspteval**, **xLS**, **yLSteval**, **a0**, **a**, **b**, **ydfteval** into Matlab Grader together with your Python code.

2- Import the following:

```

from numpy import ones, loadtxt, pi, cos, sin, log, exp, array, imag, real,
zeros, floor
from scipy.interpolate import CubicSpline
from scipy.fft import fft
from scipy.linalg import lstsq

```

The first column in the data set below contains the numbers from 1 to 11 and the second column shows the number of electric cars sold in Australia over a period of years starting from 2014 (corresponding to number 1) till 2024 (corresponding to number 11). The goal of this question is to estimate the number of electric cars sold in Australia in the first half of 2021, which corresponds to number 7.5, using three approximation methods: cubic spline, linearized least square and trigonometric polynomial approximation. The final answer from each approximation must be a positive integer, and so, you may use the floor command to round down the result.

1	1320
2	1760
3	1370
4	2300
5	3600
6	9200
7	6900
8	20400
9	38900
10	98409
11	113200

Remark: In this question, the trigonometric polynomial approximation leads to an **unrealistic** estimation.

- a) Store the first column in the column vector **tdata** and the second column in the column vector **ydata**.
- b) Compute the not-a-knot cubic spline which interpolates the given data. Evaluate the spline at $teval=7.5$ and store the results in the variable **yspteval**.
- c) Consider the least-squares fit to the given data of the form $y(t) = \alpha e^{\beta t}$. Compute the parameters (α, β) and store the results in the column vector **xLS**.
- d) Evaluate the least squares approximation at $teval$ and store the results in the variable **yLSteval**.

Let w (with entries w_i for $1 \leq i \leq m$) be the complex DFT of the data **ydata**, which can be calculated using the Matlab\Python function `fft`. The number of data points $m=11$.

- e) Calculate $a_0 = \frac{\Re w_1}{m}$, where $\Re w_1$ is the real part of w_1 , and store the result in **a0**. Hint: For Matlab users, you need to use the command 'real(x)' to find the real part of x. For Python users, you need to use the command 'x.real' to find the real part of x

- f) Calculate $a_k = \frac{2}{m} \Re w_{k+1}$ for $k = 1, 2, 3, 4, 5$, where $\Re w_{k+1}$ is the real part of w_{k+1} , and store the results in the row vector **a**.

- g) Calculate the row vector **b** with entries $b_k = -\frac{2}{m} \Im w_{k+1}$ for $k = 1, 2, 3, 4, 5$ where $\Im w_{k+1}$ is the imaginary part of w_{k+1} . Hint: For Matlab users, you need to use the command 'imag(x)' to find the imaginary part of x. For Python users, you need to use the command 'x.imag' to find the imaginary part of x.

- h) Calculate the DFT approximation

$$a_0 + \sum_{i=1}^5 a_i \cos(2\pi i teval / m) + b_i \sin(2\pi i teval / m)$$

and store the results in the variable **ydfteval**.

Matlab code:

```
tdata = data(:,1);
ydata = data(:,2);
teval=7.5;
pp = spline(tdata,ydata);
yspteval=floor(ppval(pp,teval))

A=[ones(size(tdata)) tdata];
x=A\log(ydata);
xLS=[exp(x(1));x(2)];
yLSteval=floor(xLS(1)*exp(xLS(2)*teval))

m=11;p=5;
w=fft(ydata);

b(1:p)=-2*imag(w(2:p+1))/m;
a(1:p)=2*real(w(2:p+1))/m;
a0=w(1)/m;
ydfteval=a0;
for i=1:p;
    ydfteval=ydfteval ...
+a(i)*cos(2*pi*i/m*teval)...
+b(i)*sin(2*pi*i/m*teval);
end
ydfteval=floor(ydfteval)
```

Python code:

```
from numpy import ones, loadtxt, pi,
cos, sin, log, exp, array, imag, real,
zeros, floor
from scipy.interpolate import
CubicSpline
from scipy.fft import fft
from scipy.linalg import lstsq

tdata = data[:, 0]
ydata = data[:, 1]
teval = 7.5

spline = CubicSpline(tdata, ydata)
yspteval = floor(spline(teval))

A = ones((11,2))
A[:,1]=tdata
x = lstsq(A, log(ydata))[0]
xLS = array([exp(x[0]), x[1]])
yLSteval = floor(xLS[0] * exp(xLS[1] *
teval))

m = 11, p = 5
w = fft(ydata)

b = zeros(p)
a = zeros(p)

b[:p] = -2 * imag(w[1:p+1]) / m
a[:p] = 2 * real(w[1:p+1]) / m
a0 = real(w[0]) / m

ydfteval = a0
for i in range(1, p+1):
    ydfteval += a[i-1] * cos(2 * pi * i
* teval / m) + b[i-1] * sin(2 * pi * i
* teval / m)

ydfteval=floor(ydfteval)
```

Question 5.

For Python users: 1- Copy the variables **x**, **A**, **B**, and **L**, **Uapprox**, and **RE** into Matlab Grader together with your Python code. 2- Import the following:

```
from numpy import ones, linspace, exp, zeros, inf
from numpy.linalg import solve, norm
from scipy.sparse import diags
```

In this question we use a finite difference scheme to approximate the solution of the two-point boundary value problem:

$$\frac{d^2 u}{dx^2}(x) - \frac{du}{dx}(x) - 6u(x) = 0$$

for $-1 < x < 1$, with $u(-1) = \exp(3) + \exp(-4)$ and $u(1) = \exp(2) + \exp(-1)$

a) Set $N = 6$ and $h = 2/N$.

b) Define the nodes $x_n = -1 + nh$, for $n = 1, 2, \dots, N-1$, and save them in the column vector called **x**.

c) Approximate $u(x_n)$ by U_n using the finite difference method:

$$2(U_{n+1} - 2U_n + U_{n-1}) - h(U_{n+1} - U_{n-1}) - 12h^2 U_n = 0 \text{ for } 1 \leq n \leq N-1,$$

with $U_0 = \exp(3) + \exp(-4)$ and $U_N = \exp(2) + \exp(-1)$, to obtain a system of linear

equations $(2A - hB - 12h^2C)U = L$ where A , B and C are $(N-1) \times (N-1)$ constant matrices and L is a column vector of length $N-1$, given by

$$L = [(-2-h)U_0; 0; 0; 0; (-2+h)U_N].$$

Store the matrix A in the variable **A**, the matrix B in the variable **B**, and the right-hand side L in the column vector **L**.

d) Solve the system of linear equations for U and store the results in the column vector **Uapprox**, given by the values $[U_1; U_2; U_3; U_4; U_5]$.

e) The true solution of the given boundary-value problem is:

$$u(x) = \exp(3x-1) + \exp(1-2x).$$

Define a column vector **u** of length $N-1$ with entries $u(x_1), u(x_2), \dots, u(x_{N-1})$. Use the infinity norm to compute the relative error from estimating the true vector solution **u** by the approximate vector solution **Uapprox** and store your error result in the variable **RE**.

Matlab code:

```
N = 6;
h=2/N;
x = linspace(-1+h,-1+(N-1)*h,N-1);
x=x';
d4=ones(N-2,1);
d3=ones(N-1,1);
A=diag(d4,-1)-2*diag(d3)+diag(d4,1);
B=diag(d4,1)-diag(d4,-1);
C=diag(d3);
UI=exp(3)+exp(-4);
UF=exp(2)+exp(-1);
S=2*A-h*B-12*h^2*C;
L=[(-2-h)*UI 0 0 0 (-2+h)*UF]';
Uapprox = S\L;
u = exp(3*x-1)+exp(1-2*x);
RE=norm(Uapprox-u,inf)/norm(u,inf);
```

Python code:

```
from numpy import ones, linspace, exp, zeros, inf
from numpy.linalg import solve, norm
from scipy.sparse import diags

N = 6
h = 2 / N
x = linspace(-1+h, -1+(N-1)*h, N-1)
d4 = ones(N - 2)
d3 = ones(N - 1)
A = diags([d4, -2*d3, d4], offsets=[-1, 0, 1]).toarray()
B = diags([d4, -d4], offsets=[1, -1]).toarray()
C = diags([d3], offsets=[0]).toarray()
UI = exp(3) + exp(-4)
UF = exp(2) + exp(-1)
S = 2*A - h*B - 12*h**2*C
L = zeros(N - 1)
L[0] = (-2 - h) * UI
L[-1] = (-2 + h) * UF
Uapprox = solve(S, L)
u = exp(3*x - 1) + exp(1 - 2*x)
RE = norm(Uapprox - u, inf) / norm(u, inf)
```